Automating Data Collection

Week 14 · Practice module · Textbook Chapter 14

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 16, 18 & 20, 2026

This week at a glance

We move scraping off your laptop and into the cloud — code that collects data on a schedule, unattended.

Monday | Concepts & notebook intro

- From exploration to production; why a time series beats a snapshot

Wednesday | Notebook lab

- Notebook → script → GitHub Actions: schedule it, monitor it, harden it

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 14

Companion notebook

`ch-14-automation.ipynb`

Bring a laptop

Wednesday you will push a workflow that runs itself.

1. Monday · Concepts

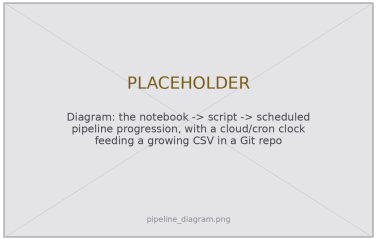
From exploration to production

Interactive notebooks are for **exploration**. Production data collection requires **automation**.

The progression is a real development workflow:

1. **Notebook —** where you develop and test
2. **Script —** a standalone `.py` that runs with no interaction
3. **Scheduled pipeline —** a script that runs itself, on the cloud, on a cadence

The logic is the *same code you already wrote*; only the environment changes. These are your existing skills, moved into production.



Same code, three environments.

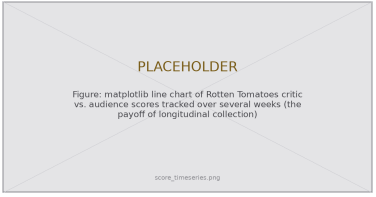
Automation: a snapshot vs. a time series

Many of the best research questions are **temporal**:

- How do Rotten Tomatoes scores evolve during a film's release?
- How does a Spotify playlist shift with the seasons?
- Which Wikipedia articles surge on a news event — and how fast does attention decay?

A scraper that runs **once** gives a cross-sectional *snapshot*. A scraper that runs **every six hours for three months** gives a **panel dataset** — trends, cycles, and responses to events.

The infrastructure is nearly free; the analytical payoff is large.



The payoff of collecting the same thing, repeatedly.

Automation raises the stakes

A manual scrape you run once makes a handful of requests. A scraper running **every hour, indefinitely**, accumulates a far larger footprint — so the ethics of Chapter 2 scale up with it.

Automation is also how you build the **ownership** the post-API chapter describes: infrastructure that runs on *your* terms and schedule, not on platform goodwill.

But ownership is **stewardship**: monitoring, maintaining, and updating scrapers is ongoing labor. Building your own data infrastructure means accepting responsibility for its upkeep.

Threads we pull forward

- **Ethics** — a bigger footprint, the same politeness rules
- **Protocols** — reuse retries & backoff for flaky runs
- **Research design** — a panel dataset you can actually analyze

In the notebook

Wednesday's companion notebook builds the pipeline one step at a time:

- **Step 1 —** convert a Rotten Tomatoes scraper from a notebook into a standalone script with headless Selenium
- **Step 2 —** wrap it in a GitHub Actions workflow that runs every six hours
- **Monitor —** catch failures and "silent successes"; harden the workflow
- **Step 3 —** read the accumulated CSV back into pandas and plot the time series

Test locally first

A `print()` you can run instantly beats debugging inside YAML, where every attempt is a commit, a push, and a wait for the runner. If it runs on your machine, it will very likely run in Actions.

2. Wednesday · Notebook Lab

Step 1 — notebook to script

```python
# scraper.py -- a standalone script; no notebook
import csv, os

FIELDNAMES = ["timestamp", "url",
              "critic_score", "audience_score"]

def get_scores(url): ... # Selenium (next slide)
def save_scores(scores): ... # append one CSV row

def main():
    save_scores(get_scores(URL))

if __name__ == "__main__": # the entry point
    main()
```

Three moves turn a notebook into production code:

- Wrap the work in a `main()` and guard it with `if __name__ == "__main__"` so it runs *only* when executed directly.
- Pin the schema in `FIELDNAMES` — a fixed column order that stays stable across every run.
- Open the CSV in **append** mode: each run adds one timestamped row.

Headless Selenium on a server

```
opts = Options()
opts.add_argument("--headless") # no window
opts.add_argument("--no-sandbox") # req'd on CI
opts.add_argument("--disable-dev-shm-usage")
driver = webdriver.Chrome(options=opts)

driver.get(url)
WebDriverWait(driver, 10).until( # let JS render
  EC.presence_of_element_located(
    (By.TAG_NAME,
     "score-icon-critic-deprecated")))
soup = BeautifulSoup(
    driver.page_source, "html.parser")
```

`-headless` runs Chrome with no window; `-no-sandbox` and `-disable-dev-shm-usage` are required on a GitHub Actions runner.

Read `page_source` *after* `WebDriverWait` — reading a JS-heavy page too early yields empty scores.

The `-deprecated` element names are the site's own web components; the suffix warns they change often. When they do, fix it in dev tools, not here.

Step 2 — schedule it with GitHub Actions

A YAML file in `.github/workflows/` runs your script on GitHub's cloud:

```yaml
# .github/workflows/scraper-action.yml
name: Scrape Rotten Tomatoes Scores
on:
  schedule:
    - cron: "0 */6 * * *" # every 6 hours (UTC)
  workflow_dispatch: # manual "Run workflow" button
permissions:
  contents: write # WITHOUT this, git push fails (403)
jobs:
  scrape:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11"}
      - run: pip install -r requirements.txt
      - uses: browser-actions/setup-chrome@v1 # matched ChromeDriver
      - run: python scraper.py
      - run: | # commit new rows
          git diff --staged --quiet || git commit -am "scores $(date -u)"
          git pull --rebase origin main && git push
```

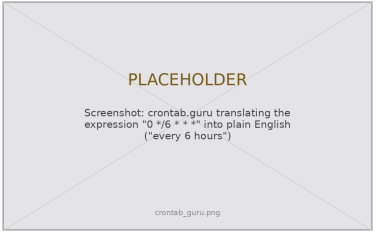
Scheduling with cron — and a Python-free option

Five fields: `minute hour day month weekday`. All times **UTC**.

Expression	Runs
`0 */6 * * *`	Every 6 hours
`0 12 * * 1-5`	Weekdays, noon
`30 8 * * *`	Daily, 8:30 AM
`0 0 1 * *`	First of month
`0 0 * * 0`	Sundays, midnight

Not every scraper needs Python — `curl` a JSON API straight from YAML:

```
- run: |
    D=$(date -u -d yesterday +%Y/%m/%d)
    curl -s ".../pageviews/top/en/$D" \
      -o data/top.json # no Python
```



`crontab.guru` translates an expression into plain English. Bookmark it.

Fail loudly: monitor & harden

The dangerous failure is the **silent success**. The run reports no error, but the CSV row is empty. The site changed and the code raised no exception. Add a validation step to find this failure. Then stop the run cleanly:

```python
def main():
    try:
        save_scores(get_scores(URL))
    except Exception as e:
        print(f"failed: {e}", file=sys.stderr)
        sys.exit(1) # mark the step FAILED
```

```yaml
- name: Validate results # in the workflow
  run: tail -1 scores.csv | grep -q ",," \
    && { echo "empty!"; exit 1; } || true
```

The Actions tab: green is not enough — read the log.

Harden further: `timeout-minutes` kills a hung job, a `concurrency` group stops overlaps, pip caching speeds runs, and retry-with-backoff (Ch. Protocols) rides out transient 503s.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the notebook:

1. Build a scraper that runs in GitHub Actions. Deploy it. Run it for 3 or more days. Then analyze the data.
2. Convert an earlier project into a `.py` script with no interface. Run the script from the terminal.
3. Design a check that monitors the output. Use the file size, the row count, or the age of the data.
4. Select a government source such as Open-Meteo or USGS. Collect data for one week. Make a visualization of the change.
5. Write `check_freshness.py`. The script must give a warning if the newest row is more than 24 hours old.
6. **5617**: run a collector for 2 or more weeks. Then write a report about the infrastructure. Use Wilson et al. (2017) as a model

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

Automation is not “set and forget.”
A scraper that runs every hour, forever,
is a promise you make to maintain it.

Ownership is stewardship.

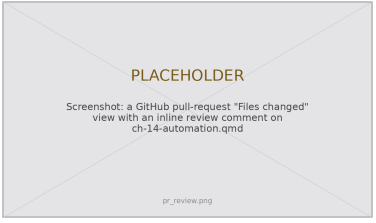
3. Friday · Textbook Revisions

Improving Chapter 14 together

Today we make the book better. Each of you opens a **pull request** against `ch-14-automation.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



High-value targets — pick one and scope it to a single PR:

- **Run the workflow live.** Fork the repo, trigger the Action, and report what broke — selector drift on the `-deprecated` elements, or a ChromeDriver mismatch. Document the fix (this is the chapter's own fragility warning).
- **Deepen "Common Issues to Debug."** Add a worked YAML-indentation failure, a `git pull -rebase` conflict walkthrough, or a `webdriver-manager` recipe for local driver drift.
- **Add a figure.** An annotated Actions run-log screenshot, the notebook→script→pipeline diagram, or a `crontab.guru` capture would each help.
- **Make an exercise turnkey.** Ship a starter `requirements.txt` + workflow template and expected output so Exercises 1 and 4 are self-checking.
- **Extend "Further Reading."** A public-interest collector (air-quality network, transit tracker) tied to the ownership callout.

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Last deck of the semester — Weeks 15–16 turn everything you've built into **research design** and your Final Project presentation (see the syllabus).

Key takeaways

1. Production collection is a progression: **notebook** → **script** → **scheduled pipeline —** the same code, moved into production.
2. Run once for a **snapshot**; run on a schedule for a **time series** (a panel dataset you can actually analyze).
3. GitHub Actions gives free cloud scheduling; set `permissions: contents: write` or `git push` 403s, and remember cron is **UTC**.
4. **Fail loudly**: validate output to catch silent successes, wrap scrapes in try/except, and harden with timeouts, concurrency, and backoff.
5. Automation is not "set and forget" — owning your data infrastructure means committing to its **stewardship**.